

Министерство образования Республики Беларусь
Учреждение образования
«БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ»

Факультет: Компьютерных систем и сетей

Кафедра: Электронных вычислительных машин

Дисциплина: Жизненный цикл разработки программного обеспечения

ОТЧЕТ
О ЛАБОРАТОРНОЙ РАБОТЕ №1
на тему
«ИНТЕГРИРОВАННАЯ СРЕДА РАЗРАБОТКИ ПО»
Вариант 15

Студент:

Д.В. Мистюкович

Преподаватель:

Д.В. Куприянова

Минск 2026

1 ЦЕЛЬ И ЗАДАЧИ

Цель работы – изучить основные возможности интегрированной среды разработки JetBrains Rider при создании объектно-ориентированного приложения на языке C#, освоить запуск и отладку программы с графическим интерфейсом, а также применить архитектурный паттерн MVC с активной моделью.

В соответствии с вариантом 15 необходимо разработать утилиту-генератор паролей. Для выполнения работы поставлены следующие задачи:

- 1 Создать и настроить проект C# в JetBrains Rider, изучить средства редактирования, сборки, запуска и отладки;
- 2 Подобрать средства Rider для работы с UML и SysML v2 и отразить результат настройки в отчёте;
- 3 Разработать графическое приложение с главной формой и отдельным диалоговым окном для ввода параметров генерации;
- 4 Обеспечить выбор длины пароля, количества паролей и используемых наборов символов (заглавные, строчные буквы, цифры, спецсимволы), сохранение последних корректных параметров и их восстановление при повторном вводе;
- 5 Реализовать проверку входных данных и вывод понятного сообщения при ошибке;
- 6 Разделить приложение на модель, представление и контроллер и реализовать активное уведомление представления моделью;
- 7 Проверить основные сценарии работы программы и подготовить ответы на контрольные вопросы.

2 КРАТКИЕ ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

2.1 Интегрированная среда разработки и её компоненты

Современная интегрированная среда разработки объединяет в одном окне редактор кода, инструменты навигации и рефакторинга, систему сборки, средства запуска и отладки, встроенный терминал и интеграцию с системами контроля версий. В данной работе используется JetBrains Rider – среда, ориентированная на платформу .NET. Rider анализирует C#-код прямо во время набора, подсвечивает ошибки и предупреждения, предлагает быстрые исправления и позволяет запускать проект с методом Main без переключения в консоль.

2.2 Редактирование и навигация в Rider

Редактор Rider поддерживает автодополнение, автоматический импорт пространств имён, форматирование, безопасное переименование и поиск использований. Основные сочетания клавиш, применяемые при разработке: Ctrl+B – переход к объявлению; Alt+F7 – поиск использований; Ctrl+Alt+L – форматирование кода; Alt+Enter – быстрые исправления и намерения. Структура решения отображается в окне Explorer, а сообщения компилятора и вывод программы – в окнах Build и Run соответственно.

2.3 Сборка, запуск и отладка

Исходный код на C# компилируется компилятором Roslyn, входящим в состав .NET SDK, в промежуточный язык IL, который выполняется средой CLR. В Rider сборка выполняется командой Build Solution, а запуск и отладка приложения – командами Run и Debug. Точка останова ставится щелчком слева от строки кода или сочетанием Ctrl+F8. При остановке окно Debug показывает стек вызовов и текущие значения переменных; пошаговое выполнение обеспечивают команды Step Over, Step Into и Step Out.

2.4 Паттерн MVC

MVC разделяет приложение на три части. Model хранит состояние и выполняет предметную логику. View формирует интерфейс и отображает данные. Controller обрабатывает действия пользователя и передаёт команды модели. Такое разделение уменьшает связанность классов и позволяет проверять логику генерации независимо от интерфейса Windows Forms.

2.5 Активная и пассивная модель

В варианте с пассивной моделью контроллер после изменения данных сам инициирует обновление представления. Активная модель сообщает

подписчикам об изменении состояния самостоятельно. В разработанном приложении класс `Model` уведомляет подписчиков о двух типах событий: `DataChanged` – при успешном сохранении настроек или завершении генерации, и `ValidationError` – при обнаружении некорректных входных данных. На событие `DataChanged` подписан метод `Form1.OnModelDataChanged`, обновляющий отображаемые параметры и список паролей; на событие `ValidationError` – метод `Form1.OnModelValidationError`, показывающий сообщение об ошибке.

3 ОПИСАНИЕ ВЫПОЛНЕННОГО ВАРИАНТА ЗАДАНИЯ

3.1 Постановка задачи

Разработано приложение, которое генерирует один или несколько случайных паролей заданной длины на основе выбранных пользователем наборов символов. Главная форма содержит кнопки «Ввести данные» и «Сгенерировать», метку `lblParamsInfo` с текущими параметрами генерации и многострочное текстовое поле `txtResults` только для чтения, в котором построчно выводятся сгенерированные пароли. Параметры генерации (длина пароля, количество паролей, использование заглавных и строчных букв, цифр и специальных символов) задаются в отдельном модальном окне `InputDialogForm`. Приложение проверяет корректность введенных данных и не допускает генерацию при отсутствии выбранных наборов символов или недопустимых числовых значениях. Пароли формируются с использованием криптографически стойкого генератора случайных чисел `System.Security.Cryptography.RandomNumberGenerator`.

3.2 Настройка среды и расширений для UML и SysML v2

В JetBrains Rider открыт проект с решением `PasswordGeneratorApp.sln` и пакетом `PasswordGeneratorApp`; в качестве Target Framework выбран `.NET 10.0 (net10.0-windows)` с включённой поддержкой `Windows Forms (UseWindowsForms)`. Класс `Program` содержит метод `Main` и используется в конфигурации запуска. Для построения UML- и SysML-диаграмм в Rider используется плагин `PlantUML Integration`, устанавливаемый через `Settings -> Plugins`; после установки диаграмма классов приложения строится вручную в файле формата `.puml` либо через пункт контекстного меню `Diagrams -> Show Diagram`.

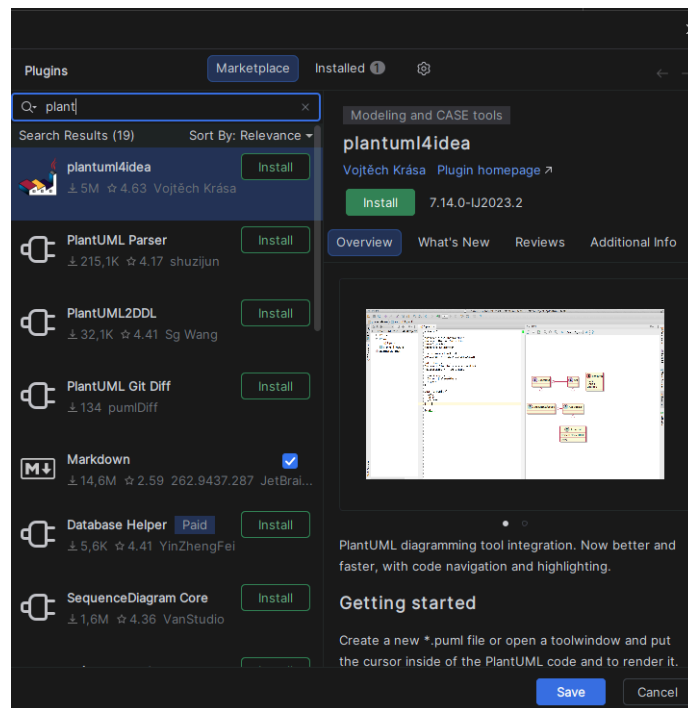


Рисунок 3.1 – Средства Rider для работы с UML и SysML v2

Структурное представление системы в нотации, близкой к SysML v2:

```
package PasswordGeneratorApp {
    part def PasswordSettings;
    part def Model;
    part def Controller;
    part def InputDialogForm;
    part def Form1;
    part application {
        part settings : PasswordSettings;
        part model : Model;
        part controller : Controller;
        part dialog : InputDialogForm;
        part view : Form1;
        connect view to controller;
        connect controller to model;
        connect controller to dialog;
        connect model to settings;
    }
}
```

3.2.1 Демонстрация отладчика

Работа отладчика демонстрируется в методе `Generate()` класса `Model`. Точка останова устанавливается на строке формирования символа пароля `password[j] = validChars[randomBytes[j] % validChars.Length]`. В окне `Debug` можно проследить, как поля `CurrentSettings.Length`, `CurrentSettings.Count` и со-

держимое строки `validChars` влияют на итоговый результат, а также проверить значения `randomBytes[j]` и вычисляемого индекса с помощью функции `Evaluate Expression`.

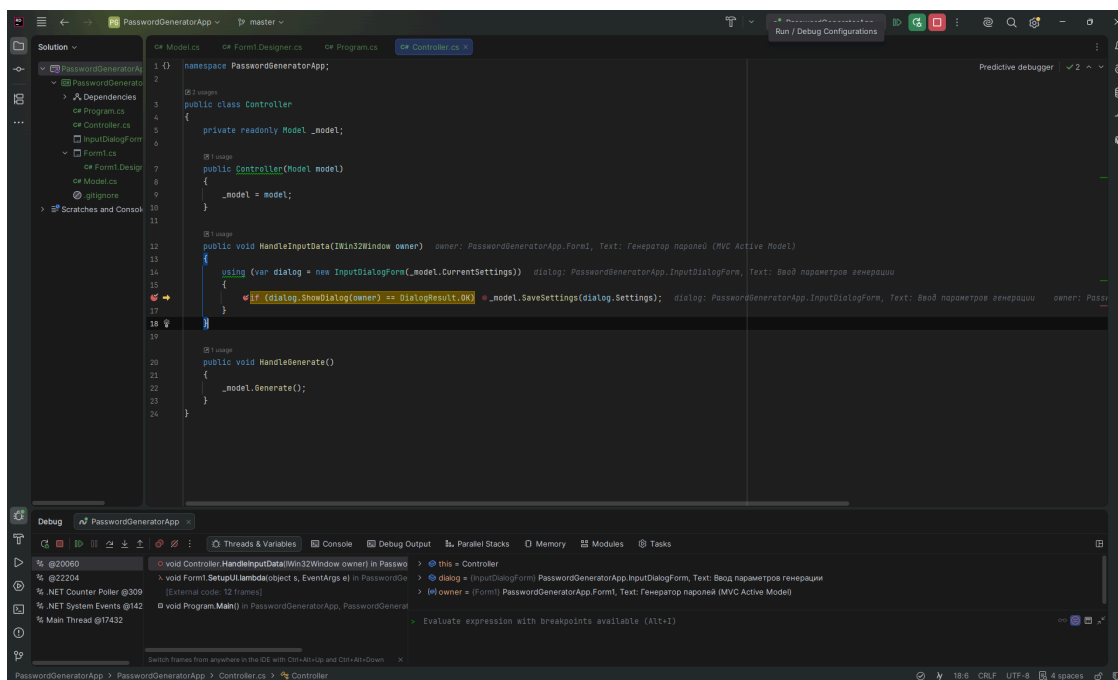


Рисунок 3.2 – Остановка программы в отладчике Rider

3.3 Структура проекта и каркас MVC с активной моделью

Проект разделён на следующие компоненты:

- `PasswordSettings` – класс-контейнер параметров генерации: длина пароля `Length`, количество паролей `Count` и флаги используемых наборов символов `IncludeUppercase`, `IncludeLowercase`, `IncludeDigits`, `IncludeSpecial`;
- `Model` – хранит текущие настройки `CurrentSettings` и список сгенерированных паролей `GeneratedPasswords`; выполняет валидацию (`ValidateSettings`), сохранение настроек (`SaveSettings`) и генерацию паролей (`Generate`); реализует активную модель, уведомляя подписчиков о своих изменениях через события `DataChanged` и `ValidationError`;
- `Controller` – обрабатывает действия пользователя: открывает диалог ввода параметров и передаёт введённые данные модели (`HandleInputData`), инициирует генерацию паролей (`HandleGenerate`);
- `InputDialogForm` – модальное окно для ввода параметров генерации: два счётчика `NumericUpDown` для длины и количества паролей и четыре флажка `CheckBox` для выбора наборов символов; при открытии окна поля заполняются текущими настройками модели, а по кнопке «Принять» формируется новый объект `PasswordSettings`;

– Form1 – главная форма приложения: создаёт экземпляры Model и Controller, подписывается на события модели, содержит элементы управления (кнопки, метку с параметрами, текстовое поле результатов) и обновляет их при получении уведомлений от модели;

– Program – точка входа приложения, вызывающая ApplicationConfiguration.Initialize() и Application.Run(new Form1()).

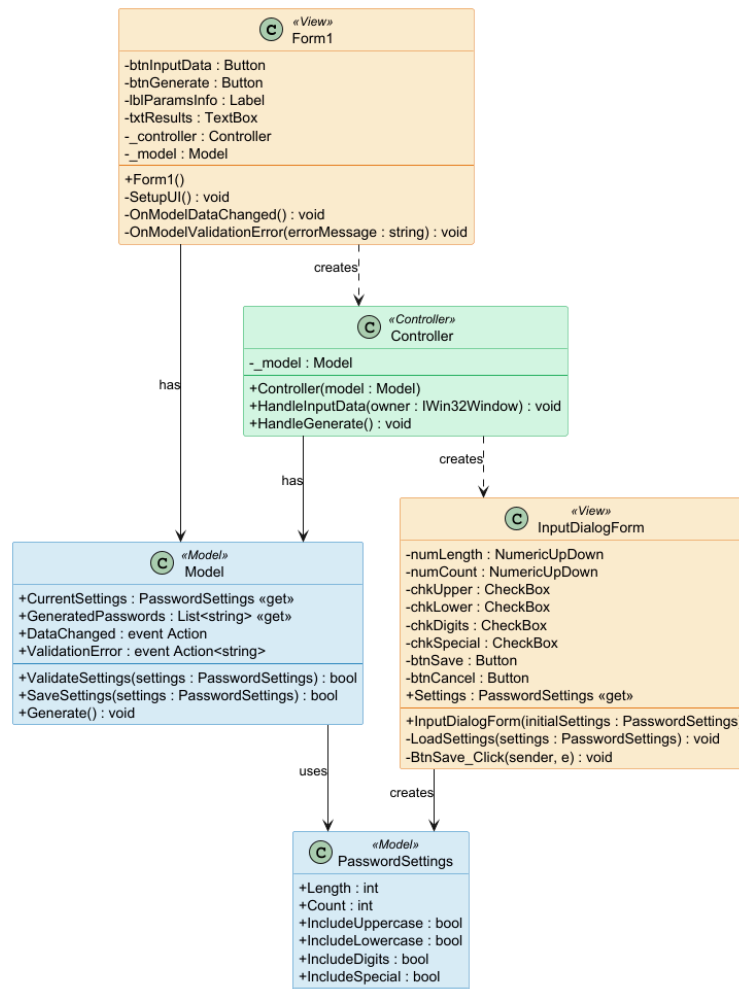


Рисунок 3.3 – Диаграмма типов и интерфейсов приложения

3.4 Модель

Model реализует активную модель. Метод `ValidateSettings` проверяет, что длина пароля и количество паролей больше нуля и что выбран хотя бы один набор символов; при нарушении любого из условий вызывается событие `ValidationError` с текстом ошибки, а генерация или сохранение настроек прерывается. Метод `SaveSettings` вызывает `ValidateSettings` и, если проверка пройдена, обновляет `CurrentSettings` и вызывает событие `DataChanged`. Метод `Generate()` повторно проверяет `CurrentSettings`, формирует строку допустимых символов `validChars` из включённых наборов (строчные и заглавные буквы, цифры, спецсимволы), затем для каждого из `Count` паролей с помощью

RandomNumberGenerator.GetBytes заполняет массив случайных байтов и по каждому байту выбирает символ из validChars по остатку от деления на длину строки; сформированные пароли добавляются в список GeneratedPasswords, после чего вызывается событие DataChanged.

3.5 Представление

Form1 отображает текущие параметры генерации и список сгенерированных паролей. Метод OnModelDataChanged, подписанный на событие DataChanged, формирует текст метки lblParamsInfo (длина, количество и перечень активных наборов символов) и заполняет текстовое поле txtResults строками из GeneratedPasswords. Метод OnModelValidationError, подписанный на событие ValidationError, показывает пользователю сообщение об ошибке через MessageBox.Show. Кнопка «Ввести данные» открывает диалоговое окно параметров, кнопка «Сгенерировать» запускает генерацию паролей по текущим сохранённым настройкам.

3.6 Контроллер

Controller является посредником между представлением и моделью. Метод HandleInputData создаёт InputDialogForm, передавая ему текущие настройки модели CurrentSettings, отображает диалог модально и, если пользователь подтвердил ввод (DialogResult.OK), передаёт полученный объект PasswordSettings в модель через SaveSettings; модель самостоятельно валидирует данные и уведомляет представление. Метод HandleGenerate вызывает Model.Generate() без дополнительных параметров, поскольку модель уже хранит актуальные настройки.

3.7 Запоминание данных и обработка ошибок

Приложение предотвращает потерю корректных настроек: при повторном открытии окна параметров метод InputDialogForm.LoadSettings заполняет все поля значениями CurrentSettings модели, поэтому пользователь видит последние сохранённые параметры, а не пустую форму. Элементы NumericUpDown ограничивают ввод длины (1–128) и количества паролей (1–100) на уровне интерфейса. Дополнительная проверка выполняется в модели: если длина или количество не больше нуля либо не выбран ни один набор символов, метод ValidateSettings вызывает событие ValidationError с описанием причины («Длина пароля должна быть больше 0.», «Количество паролей должно быть больше 0.», «Выберите хотя бы один набор символов.»), а Form1 показывает это сообщение пользователю через MessageBox; при этом CurrentSettings не изменяется, и ранее сохранённые корректные значения не теряются.

4 РЕЗУЛЬТАТЫ РАБОТЫ

В JetBrains Rider создано и проверено приложение на C# для варианта 15. Программа генерирует заданное количество паролей случайной длины с использованием выбранных наборов символов, сохраняет последние корректные параметры генерации и сообщает о некорректном вводе. Архитектура соответствует MVC с активной моделью: контроллер обрабатывает действия пользователя, модель Model выполняет валидацию и генерацию паролей и уведомляет представление о результате через события `DataChanged` и `ValidationError`.

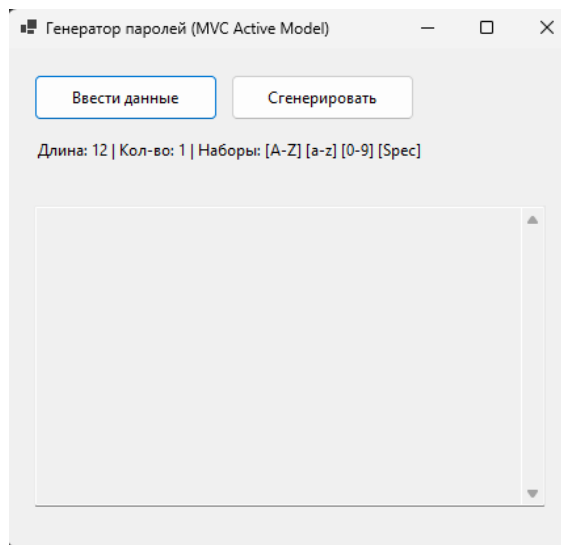


Рисунок 4.1 – Главная форма после запуска

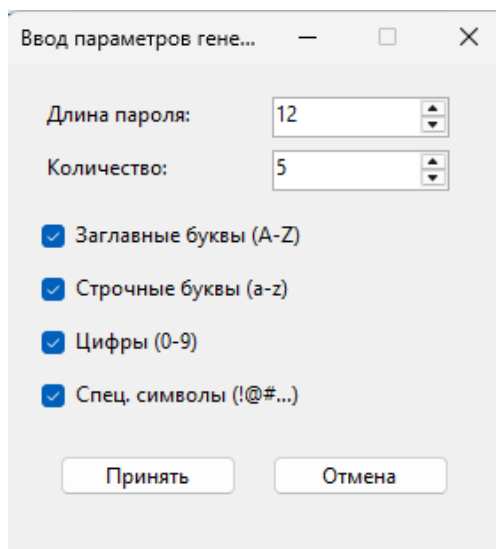


Рисунок 4.2 – Окно ввода параметров

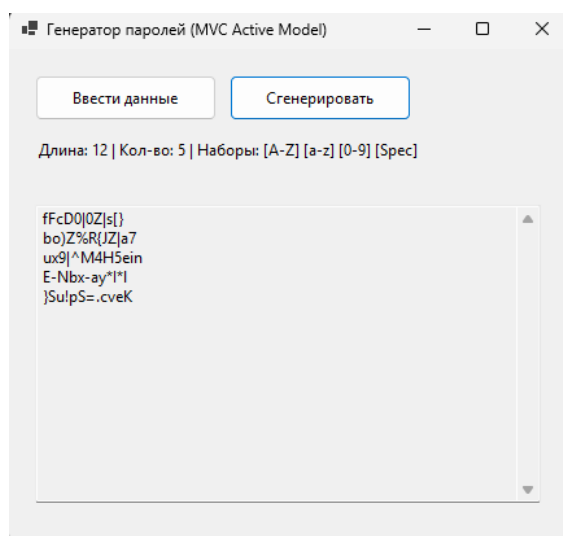


Рисунок 4.3 – Результат генерации паролей

5 ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

Вопрос 1. Какие компоненты входят в состав IDE?

В состав IDE входят редактор кода, компилятор или интерпретатор, система сборки и запуска, отладчик, средства навигации и рефакторинга, терминал, интеграция с системой контроля версий и механизм расширений (плагинов). В JetBrains Rider основные рабочие окна называются Explorer, Build, Run, Debug, Problems и Terminal.

Вопрос 2. Каковы основные приёмы работы с редактором кода?

К основным приёмам относятся автодополнение, быстрые исправления, автоматический импорт пространств имён, форматирование, переход к объявлению, поиск использований, переименование, просмотр иерархии классов и анализ предупреждений. Эти операции ускоряют написание кода и уменьшают вероятность механических ошибок.

Вопрос 3. Каким образом выполняется сборка и запуск проекта?

JetBrains Rider собирает C#-файлы командой Build Solution. Для запуска выбирается проект с классом Program и методом [STAThread] static void Main(), и выполняется команда Run; в режиме Debug программа работает под управлением отладчика и останавливается на заданных точках. Проект также можно собрать из терминала командой dotnet build и запустить командой dotnet run.

Вопрос 4. Что такое MVC?

MVC – архитектурный паттерн, который разделяет данные и предметную логику, графический интерфейс и обработку действий пользователя. В данной работе роль модели выполняет класс Model, роль представления – Form1 и InputDialogForm, а роль контроллера – класс Controller.

Вопрос 5. Чем отличается активная модель от пассивной?

Пассивная модель не уведомляет представление о своих изменениях, поэтому после изменения данных контроллер должен отдельно обновлять интерфейс. Активная модель публикует событие при изменении своего состояния. В приложении класс Model вызывает события DataChanged и ValidationError, а Form1 получает уведомления через обработчики OnModelDataChanged и OnModelValidationError и самостоятельно обновляет элементы формы.

ЗАКЛЮЧЕНИЕ

В ходе лабораторной работы изучены основные возможности JetBrains Rider для редактирования, сборки, запуска и отладки приложения на C#, а также средства работы с UML- и SysML-диаграммами.

Для варианта 15 разработана утилита генерации паролей с графическим интерфейсом Windows Forms. Программа поддерживает настройку длины и количества паролей, выбор наборов символов, проверку корректности введённых данных и криптографически стойкую генерацию с помощью RandomNumberGenerator. Компоненты приложения разделены по паттерну MVC, а активная модель Model уведомляет представление о новом состоянии с помощью событий DataChanged и ValidationError.

ПРИЛОЖЕНИЕ А
(обязательное)
Листинг исходного кода приложения

Program.cs

```
namespace PasswordGeneratorApp;

internal static class Program
{
    [STAThread]
    private static void Main()
    {
        ApplicationConfiguration.Initialize();
        Application.Run(new Form1());
    }
}
```

Model.cs

```
using System.Security.Cryptography;

namespace PasswordGeneratorApp;

public class PasswordSettings
{
    public int Length { get; set; } = 12;
    public int Count { get; set; } = 1;
    public bool IncludeUppercase { get; set; } = true;
    public bool IncludeLowercase { get; set; } = true;
    public bool IncludeDigits { get; set; } = true;
    public bool IncludeSpecial { get; set; } = true;
}

public class Model
{
    private const string LowercaseChars =
"abcdefghijklmnopqrstuvwxyz";
    private const string UppercaseChars =
"ABCDEFGHIJKLMNOPQRSTUVWXYZ";
    private const string DigitChars = "0123456789";
    private const string SpecialChars = "!@#$%^&*()_+=[{}|;:,.<>?";
    public PasswordSettings CurrentSettings { get; private set; } =
new();
    public List<string> GeneratedPasswords { get; } = new();

    public event Action DataChanged;
```

```

public event Action<string> ValidationError;

public bool ValidateSettings(PasswordSettings settings)
{
    if (settings.Length <= 0)
    {
        ValidationError?.Invoke("Длина пароля должна быть больше
0.");
        return false;
    }

    if (settings.Count <= 0)
    {
        ValidationError?.Invoke("Количество паролей должно быть
больше 0.");
        return false;
    }

    if (!settings.IncludeUppercase && !settings.IncludeLowercase
&&
        !settings.IncludeDigits && !settings.IncludeSpecial)
    {
        ValidationError?.Invoke("Выберите хотя бы один набор
СИМВОЛОВ.");
        return false;
    }

    return true;
}

public bool SaveSettings(PasswordSettings settings)
{
    if (!ValidateSettings(settings))
        return false;

    CurrentSettings = settings;
    DataChanged?.Invoke();
    return true;
}

public void Generate()
{
    if (!ValidateSettings(CurrentSettings))
        return;

    GeneratedPasswords.Clear();

    var validChars = "";

```

```

        if (CurrentSettings.IncludeLowercase) validChars +=
LowercaseChars;
        if (CurrentSettings.IncludeUppercase) validChars +=
UppercaseChars;
        if (CurrentSettings.IncludeDigits) validChars += DigitChars;
        if (CurrentSettings.IncludeSpecial) validChars +=
SpecialChars;

        using (var rng = RandomNumberGenerator.Create())
        {
            for (var i = 0; i < CurrentSettings.Count; i++)
            {
                var password = new char[CurrentSettings.Length];
                var randomBytes = new byte[CurrentSettings.Length];
                rng.GetBytes(randomBytes);

                for (var j = 0; j < CurrentSettings.Length; j++)
                    password[j] = validChars[randomBytes[j] %
validChars.Length];

                GeneratedPasswords.Add(new string(password));
            }
        }

        DataChanged?.Invoke();
    }
}

```

Controller.cs

```

namespace PasswordGeneratorApp;

public class Controller
{
    private readonly Model _model;

    public Controller(Model model)
    {
        _model = model;
    }

    public void HandleInputData(IWin32Window owner)
    {
        using (var dialog = new
InputDialogForm(_model.CurrentSettings))
        {
            if (dialog.ShowDialog(owner) == DialogResult.OK)

```

```

_model.SaveSettings(dialog.Settings);
    }
}

public void HandleGenerate()
{
    _model.Generate();
}
}

```

Form1.cs

```

namespace PasswordGeneratorApp;

public partial class Form1 : Form
{
    private readonly Controller _controller;
    private readonly Model _model;
    private Button btnGenerate;

    private Button btnInputData;
    private Label lblParamsInfo;
    private TextBox txtResults;

    public Form1()
    {
        InitializeComponent();

        SetupUI();

        _model = new Model();
        _controller = new Controller(_model);

        _model.DataChanged += OnModelDataChanged;
        _model.ValidationError += OnModelValidationError;

        // Отображаем начальное состояние параметров
        OnModelDataChanged();
    }

    private void SetupUI()
    {
        Text = "Генератор паролей (MVC Active Model)";
        Size = new Size(450, 420);
        StartPosition = FormStartPosition.CenterScreen;

        btnInputData = new Button

```

```

    {
        Text = "Ввести данные",
        Left = 20,
        Top = 20,
        Width = 140,
        Height = 35
    };
    btnInputData.Click += (s, e) =>
        _controller.HandleInputData(this);

    btnGenerate = new Button
    {
        Text = "Сгенерировать",
        Left = 170,
        Top = 20,
        Width = 140,
        Height = 35
    };
    btnGenerate.Click += (s, e) => _controller.HandleGenerate();

    lblParamsInfo = new Label
    {
        Left = 20,
        Top = 70,
        Width = 400,
        Height = 40,
        Text = "Параметры: данные еще не введены."
    };

    txtResults = new TextBox
    {
        Left = 20,
        Top = 120,
        Width = 390,
        Height = 230,
        Multiline = true,
        ScrollBars = ScrollBars.Vertical,
        ReadOnly = true
    };

    Controls.Add(btnInputData);
    Controls.Add(btnGenerate);
    Controls.Add(lblParamsInfo);
    Controls.Add(txtResults);
}

private void OnModelDataChanged()
{

```

```

        var s = _model.CurrentSettings;

        lblParamsInfo.Text = $"Длина: {s.Length} | Кол-во: {s.Count}
| " +
                                $"Наборы: {(s.IncludeUppercase ? "[A-Z]
" : "")}" +
                                $"{(s.IncludeLowercase ? "[a-z] " :
""})}" +
                                $"{(s.IncludeDigits ? "[0-9] " : "")}" +
                                $"{(s.IncludeSpecial ? "[Spec] " :
"")}";

        txtResults.Lines = _model.GeneratedPasswords.ToArray();
    }

    private void OnModelValidationError(string errorMessage)
    {
        MessageBox.Show(errorMessage, "Ошибка ввода",
        MessageBoxButtons.OK, MessageBoxIcon.Error);
    }
}

```

Form1.Designer.cs

```

namespace PasswordGeneratorApp;

partial class Form1
{
    /// <summary>
    /// Required designer variable.
    /// </summary>
    private System.ComponentModel.IContainer components = null;

    /// <summary>
    /// Clean up any resources being used.
    /// </summary>
    /// <param name="disposing">true if managed resources should be
disposed; otherwise, false.</param>
    protected override void Dispose(bool disposing)
    {
        if (disposing && (components != null))
        {
            components.Dispose();
        }

        base.Dispose(disposing);
    }
}

```

```

#region Windows Form Designer generated code

/// <summary>
///   Required method for Designer support - do not modify
///   the contents of this method with the code editor.
/// </summary>
private void InitializeComponent()
{
    components = new System.ComponentModel.Container();
    AutoScaleMode = AutoScaleMode.Font;
    ClientSize = new Size(800, 450);
    Text = "Form1";
}

#endregion
}

```

InputDialogForm.cs

```

namespace PasswordGeneratorApp;

public class InputDialogForm : Form
{
    private Button btnCancel;
    private Button btnSave;
    private CheckBox chkDigits;
    private CheckBox chkLower;
    private CheckBox chkSpecial;
    private CheckBox chkUpper;
    private NumericUpDown numCount;
    private NumericUpDown numLength;

    public InputDialogForm(PasswordSettings initialSettings)
    {
        InitializeComponent();
        LoadSettings(initialSettings);
    }

    public PasswordSettings Settings { get; private set; }

    private void InitializeComponent()
    {
        Text = "Ввод параметров генерации";
        Size = new Size(300, 320);
        FormBorderStyle = FormBorderStyle.FixedDialog;
        MaximizeBox = false;
    }
}

```

```

        StartPosition = FormStartPosition.CenterParent;

        var lblLength = new Label { Text = "Длина пароля:", Left =
20, Top = 20, Width = 120 };
        numLength = new NumericUpDown { Left = 150, Top = 18, Width =
100, Minimum = 1, Maximum = 128 };

        var lblCount = new Label { Text = "Количество:", Left = 20,
Top = 50, Width = 120 };
        numCount = new NumericUpDown { Left = 150, Top = 48, Width =
100, Minimum = 1, Maximum = 100 };

        chkUpper = new CheckBox { Text = "Заглавные буквы (A-Z)",
Left = 20, Top = 85, Width = 200 };
        chkLower = new CheckBox { Text = "Строчные буквы (a-z)", Left
= 20, Top = 115, Width = 200 };
        chkDigits = new CheckBox { Text = "Цифры (0-9)", Left = 20,
Top = 145, Width = 200 };
        chkSpecial = new CheckBox { Text = "Спец. символы (!@#...)",
Left = 20, Top = 175, Width = 200 };

        btnSave = new Button { Text = "Принять", Left = 30, Top =
220, Width = 100, DialogResult = DialogResult.OK };
        btnCancel = new Button
        { Text = "Отмена", Left = 150, Top = 220, Width = 100,
DialogResult = DialogResult.Cancel };

        btnSave.Click += BtnSave_Click;

        Controls.AddRange(lblLength, numLength, lblCount, numCount,
chkUpper, chkLower, chkDigits, chkSpecial, btnSave,
        btnCancel);
    }

    private void LoadSettings(PasswordSettings settings)
    {
        numLength.Value = settings.Length;
        numCount.Value = settings.Count;
        chkUpper.Checked = settings.IncludeUppercase;
        chkLower.Checked = settings.IncludeLowercase;
        chkDigits.Checked = settings.IncludeDigits;
        chkSpecial.Checked = settings.IncludeSpecial;
    }

    private void BtnSave_Click(object sender, EventArgs e)
    {
        Settings = new PasswordSettings
        {

```

```
        Length = (int)numLength.Value,  
        Count = (int)numCount.Value,  
        IncludeUppercase = chkUpper.Checked,  
        IncludeLowercase = chkLower.Checked,  
        IncludeDigits = chkDigits.Checked,  
        IncludeSpecial = chkSpecial.Checked  
    };  
}  
}
```